

A Practical Introduction to Pyrobot and Some Common Robot Sensors

Nik Swoboda

`nswoboda@fi.upm.es`

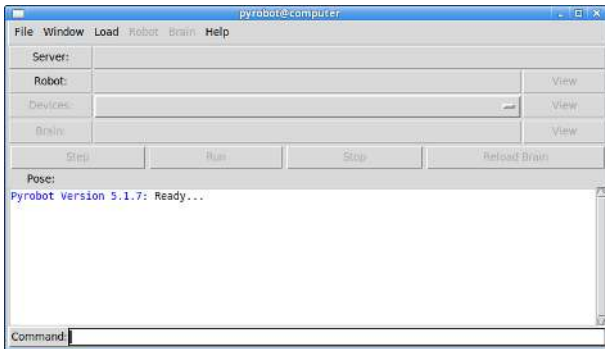
Robótica y Percepción Computacional
Universidad Politécnica de Madrid

February 10, 2026

In this lecture I will present:

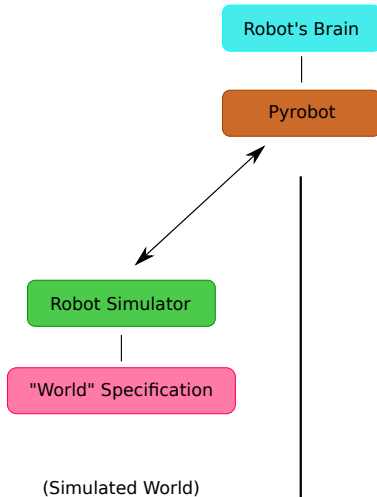
- An overview of Pyrobot
- A brief introduction to some common types of sensors used by robots
- Some answers to FAQ's regarding Python programming

Getting Started ... Pyrobot

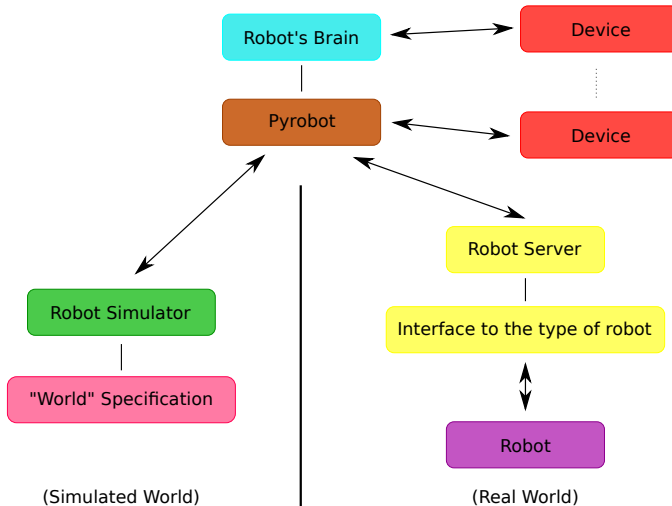


Some more info: <https://cs.brynmawr.edu/BeyondLegos/PyroModules/Introduction/index.html> and <https://github.com/dsblank/pyrobot>

Understanding How Pyrobot Works



Understanding How Pyrobot Works



Starting Up Pyrobot with the Stage Simulator

After executing pyrobot you need to select:

- Server: `StageSimulator.py`
then you also must select a world file, use `challenge.world`
- Robot: `Stage.py`
- Brain: `Avoid.py`
(this is an example, you can use any brain you like)

or, to save time, you can start things from the command line (or make a simple shell script):

```
pyrobot -s StageSimulator.py -w challenge.world \  
-r Stage.py -b Avoid.py
```

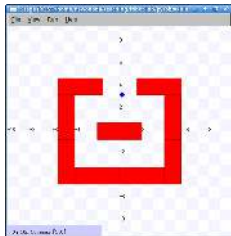
Some things to try when experimenting with Pyrobot:

- Step, Run, Stop
- Use the Joystick (in the Robot menu)
- Drag the robot
- The command line (we will use this more later), try:

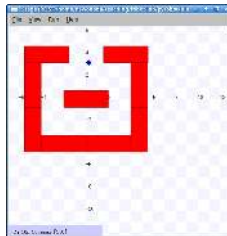
```
print(self.robot.x)
```
- ...

The Stage Simulator

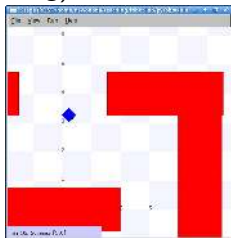
The default view from above



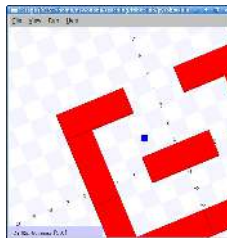
Drag the ground to scroll the world



Mouse wheel (or middle button drag) for zoom

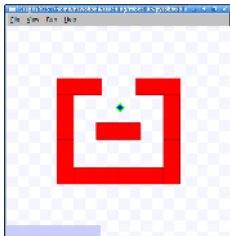


Right button drag to rotate

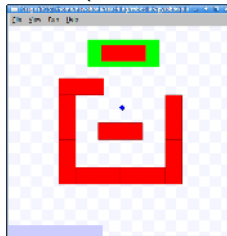


The Stage Simulator (Cont.)

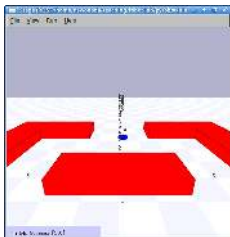
You can drag/rotate the robot



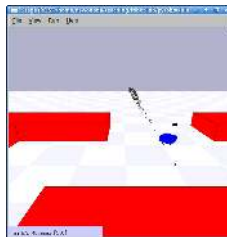
You can also move parts of the world (if they are not locked)



Press r to change to perspective view

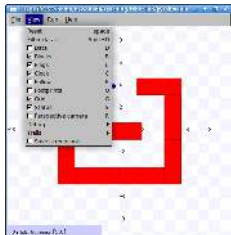


Again, you can scroll, rotate and zoom

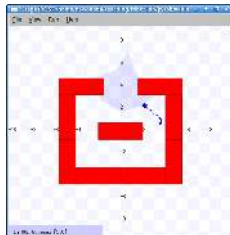


The Stage Simulator (Cont.)

More options available in the View menu



For example, visualize sonar and trail.



Stage web page: <http://playerstage.sourceforge.net/index.php?src=stage>

Information regarding Stage world files:
<https://player-stage-manual.readthedocs.io/en/stable/>

The Basic Anatomy of the Robot's Brain

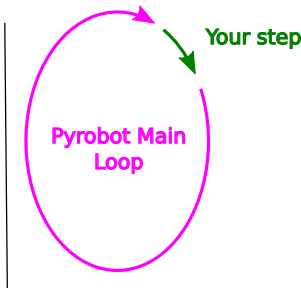
```
from pyrobot.brain import Brain

class SimpleBrain(Brain):
    # class methods

    def setup(self):
        # Optional: run once at start

    def step(self):
        # Required: run once in each iteration of the
        # main loop

def INIT(engine):
    assert (engine.robot.requires("range-sensor") and
            engine.robot.requires("continuous-movement"))
    return SimpleBrain('SimpleBrain', engine)
```



The General Pyrobot Philosophy

The general goal of Pyrobot is to provide a simple and uniform interface to many different kinds of robots.

To do this it makes use of abstractions. For example ...

Moving the Robot

The robot's `move` method is used to move the robot.

(Independent of the type of robot, we assume that it can perform continuous movement forward/backward and rotating left/right. We leave the details of achieving those movements to the drivers.)

`move` takes two arguments:

- Translation - Forward (+1.0) and backward movement (-1.0)
- Rotation - clockwise (-1.0) and counter-clockwise rotation (+1.0)

(The signs of both arguments follow the right hand rule.)

Moving the Robot (Cont.)

An example:

```
self.robot.move(1.0, -0.5) # full speed forward and half  
                           # speed rotate to right
```

In order to make working with real robots safer, the robot stops moving after a short time.

Calls to `move` in each of the executions of your step will move the robot smoothly and the robot will stop if your code crashes.

(You can try to execute a `move` command using the command line.)

Sensors - Sonar

Active Sonar (SOund NAvigation and Ranging) sensors measure distances by emitting short bursts of sound, a ping, (often ultrasonic) and measuring the time taken for that sound to return (i.e., the echo after being reflected off of an object).

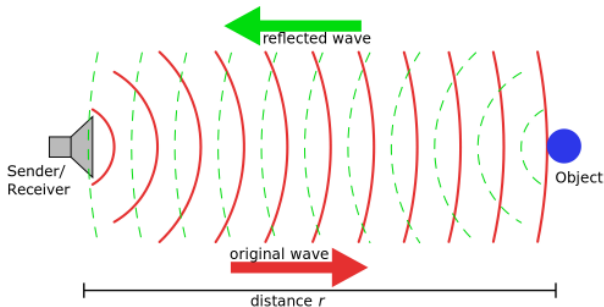
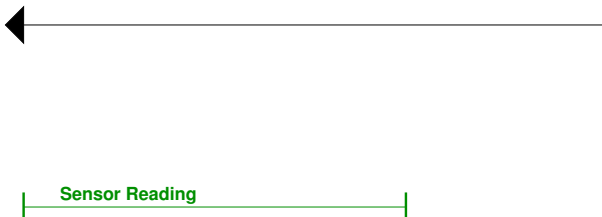


Image from <http://en.wikipedia.org/wiki/Sonar>

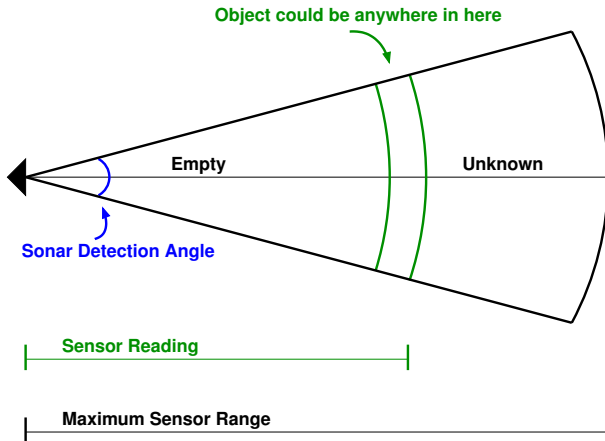
Sensors - Sonar (Cont.)

How can you interpret a sonar reading?



Sensors - Sonar (Cont.)

One reading gives the following information (there is a lot of uncertainty):



Sensors - Sonar (Cont.)

In reality things are not “black and white”



(Run demo.)

Sensors - Sonar (Cont.)

The Pioneer-3-DX's Polaroid Sonar (effective angle of about 30°)

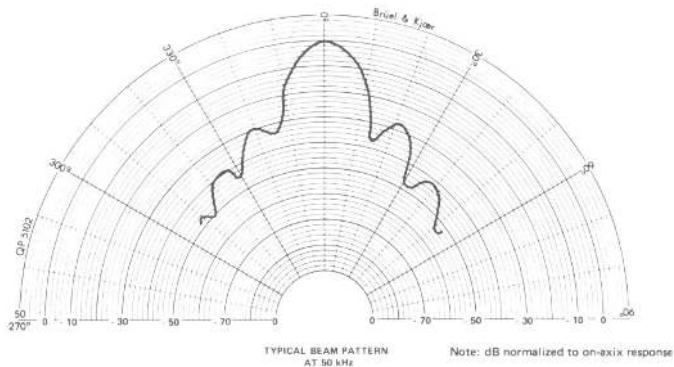
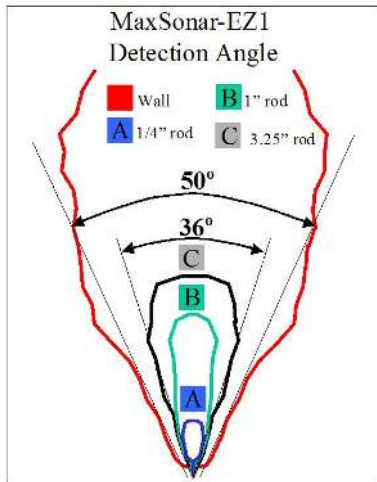


Image from *The Arc-Transversal Median Algorithm: an Approach to Increasing Ultrasonic Sensor Accuracy* by Keiji Nagatani et al.

Sensors - Sonar (Cont.)

The Erratic's sonar

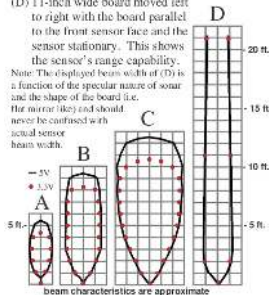


Beam Characteristics

Sample results for measured beam patterns are shown below on a 12-inch grid. The detection pattern is shown for;

- (A) 0.25-inch diameter dowel, note the narrow beam for close small objects.
- (B) 1-inch diameter dowel, note the long narrow detection pattern.
- (C) 3.25-inch diameter rod, note the long controlled detection pattern.
- (D) 11-inch wide board moved left to right with the board parallel to the front sensor face and the sensor stationary. This shows the sensor's range capability.

Note: The displayed beam width of (D) is a function of the specular nature of sonar and the shape of the board (i.e. flat mirror like) and should never be confused with actual sensor beam width.



A Popular Application of Sonar Sensors

Parking Sensor:

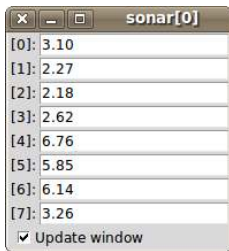


Image from https://en.wikipedia.org/wiki/File:VW_Golf_VII_-_Parking_sensor_02.jpg

Sensors - Sonar (Cont.)

Here we will use the range abstraction (which in our case corresponds to `sonar[0]`).

By selecting Devices: `sonar[0]` and then clicking on View we can see the current sensor readings:



(Use the joystick to move the robot and observe how the readings change.)

We can access these readings in the Brain using:

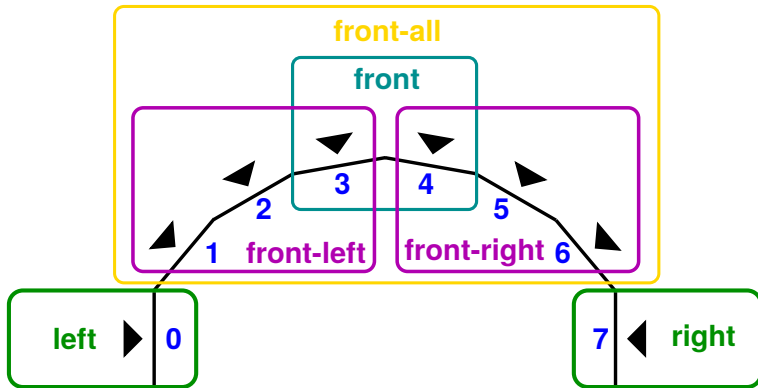
```
self.robot.range[3].distance()
```

(Try `print(self.robot.range[3].distance())` in the command line.)

Both the simulated and the real robots that we will use have 8 sonar sensors, numbered from left (0) to right (7).

Sensors - Sonar (Cont.)

Some groupings of sensors have been provided for convenience:



For example:

```
front = min([s.distance() for s in self.robot.range['front']])
```

The robot's sensor units are customizable, here are some of the choices available:

- METERS
- CM
- MM
- ROBOTS (proportional to the robot's diameter, with the diameter being 1)

By default METERS is used, but that can be changed:

```
self.robot.range.units = 'ROBOTS'
```

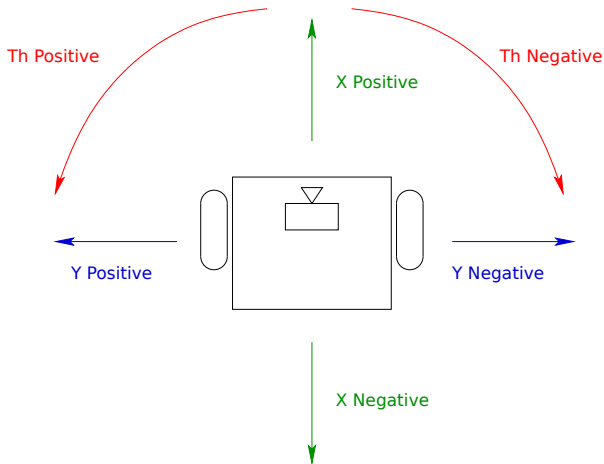
The robot's motors have sensors which calculate the number of degrees of rotation. Knowing the shape of the wheel and the timing of the rotations of both (in the case of 2 wheel robots), the robot's driver estimates its current position.

```
self.robot.x  
self.robot.y  
self.robot.th # in degrees  
self.robot.thr # in radians
```

These values are shown in the Pose: line of Pyrobot. (Signs are explained in the next transparency.)

Note: when working with the real robots, these values are *relative* to the starting position of the robot. They are *absolute* if you are using the Stage simulator.

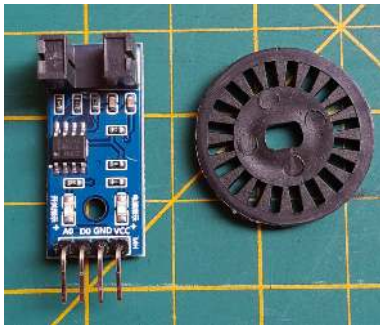
Sensors - Odometry



These values are relative to the starting position of the robot. *In stage*, the values are absolute with X Positive to the right.

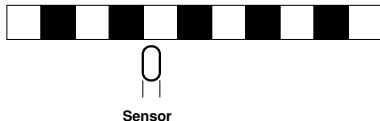
Sensors - Odometry (Cont.)

Optical wheel encoder (FC-03 and disk with 20 holes/revolution)



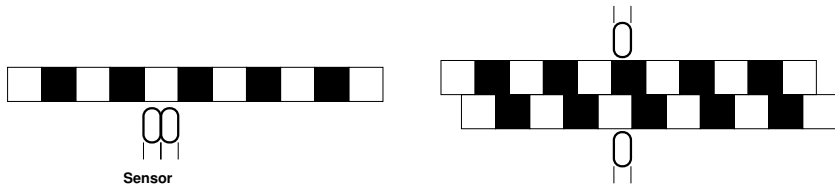
Sensors - Odometry (Cont.)

Using just one sensor results in a problem...



We do not know the direction of the rotation.

Solution: Incremental or quadrature encoder (two different kinds):



Sensors - Odometry (Cont.)

Pololu wheel encoder with 2 infrared reflectance sensors and 12 teeth (48 counts/rev, about 3mm/count)

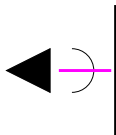


Images from <https://www.pololu.com/product/1217>

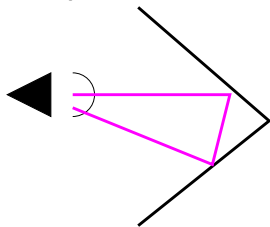
When using these sensors (odometry and sonar) in the simulation the data returned is pretty reliable (though a bit noisy).

This will NOT be the case when you start to work with the real robots.

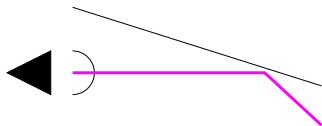
Obstacle too close



Multiple reflections



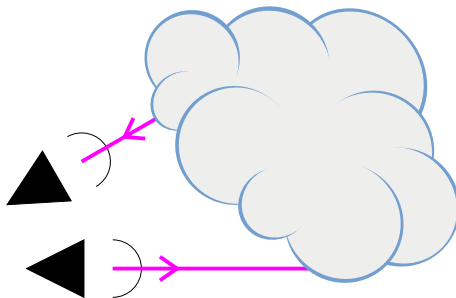
Angle too shallow



Reflection too weak



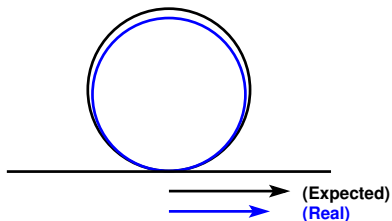
Crosstalk



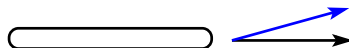
Cloud image from https://commons.wikimedia.org/wiki/File:Cartoon_cloud.svg

Odometry Error

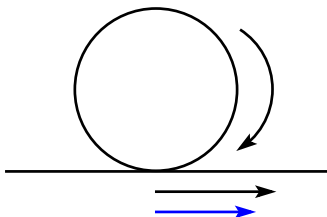
Error in wheel diameter



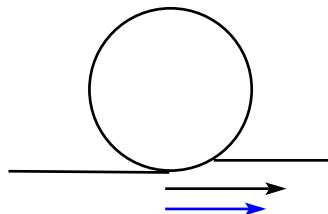
Wheel skidding



Wheel slipping



Ground compaction



Odometry Error

Very little compaction error occurs on very hard surfaces, a lot can occur on very soft surfaces.



Images from [https://commons.wikimedia.org/wiki/File:2018-06-19_\(146\)_Wheel_of_33_53_5301_485-5_at_Bahnhof_Herzogenburg.jpg](https://commons.wikimedia.org/wiki/File:2018-06-19_(146)_Wheel_of_33_53_5301_485-5_at_Bahnhof_Herzogenburg.jpg) and https://commons.wikimedia.org/wiki/File:DumontDunes_2005-04-25_paddle-tire.jpg

Sensors - Video Camera (Real Robot)

Any python code which can directly access a USB camera should work, you are welcome to try it.

But if you are curious you can also see the video captured by the robot by running an application which can capture video from a USB camera (like vlc).

Some other common robot sensors (which we will not be using)

- Touch - Bumper
- Sound - Microphone
- Acceleration - accelerometers and gyroscopes
- Inclination - inclinometers and gyroscopes
- Magnetism - compass
- Position - GPS
- Smell - chemical sensor
- Temperature, Altitude, Air Pressure

We have been talking about sensors which gather information regarding the environment of the robot. These are also known as *exteroceptive sensors*.

Sensors are also used to gather information regarding the internal state of the robot. These are also known as *proprioceptive sensors*.

Some examples:

- Battery level
- Joint or wheel positions
- Cargo weight
- ...

Some Python FAQ's

Question: Why are there no ; or { } ?

Answer: White space is meaningful in Python and it is used to define blocks of code. These two pieces of code are not the same.

```
if (frontClear):  
    # move forward  
    if (leftClear):  
        # rotate left
```

```
if (frontClear):  
    # move forward  
if (leftClear):  
    # rotate left
```

Some Python FAQ's (Cont.)

Question: Where are the variable declarations and the types?

Answer: Variables don't have to be declared and their types are derived from the data assigned to them.

```
steps = 100           # An integer assignment
meters = 1000.0        # A floating point
markerName = "cars"    # A string
```

Some Python FAQ's (Cont.)

Question: What is “self”?

Answer: “self” is a convention, and is used as the first argument of class methods to refer to the object upon which the call is being made

```
class Bag:
    def __init__(self):
        self.data = []
    def add(self, x):
        self.data.append(x)
    def addtwice(self, x):
        self.add(x)
        self.add(x)
```

Example class from <http://docs.python.org/tutorial/classes.html>

An Exercise - A Wall Following Robot

First install Pyrobot - the easiest option is to use a virtual machine with everything pre-installed, instructions are in Moodle.

The goal of the exercise is to write the `Brain` for a wall following robot.

The simplest way to do this is to write a robot controller that is purely *reactive*. I.e., one that simply considers its current surroundings (using its sonar) and reacts based upon this information.

An Exercise - A Wall Following Robot (Cont.)

Some hints for writing a reactive controller:

- Start with a copy of the `Avoid.py` brain and make modifications (the file is in `$HOME/pyrobot/plugins/brains/Avoid.py`)
- Decide whether you want your robot to follow walls clockwise or counter-clockwise.
- If you are using the `challenge.world` world, it is easier to start experimenting with following the outer walls and then to try your code with the inner walls.
- Make a short list of 'states' for your robot:
 - Something to the left
 - Something in front
 - ...
- Think about how your robot will know what state it is in, what your robot should do in each of those states and when it should change state